Московский Авиационный Институт

(Национальный Исследовательский Университет)

Институт №8 “Компьютерные науки и прикладная математика”

Кафедра №806 “Вычислительная математика и программирование”

Лабораторная работа №1 по курсу

«Операционные системы»

Группа: М8О-210БВ-25

Студент: Брандыс В.Е.

Преподаватель: Бахарев В.Д.

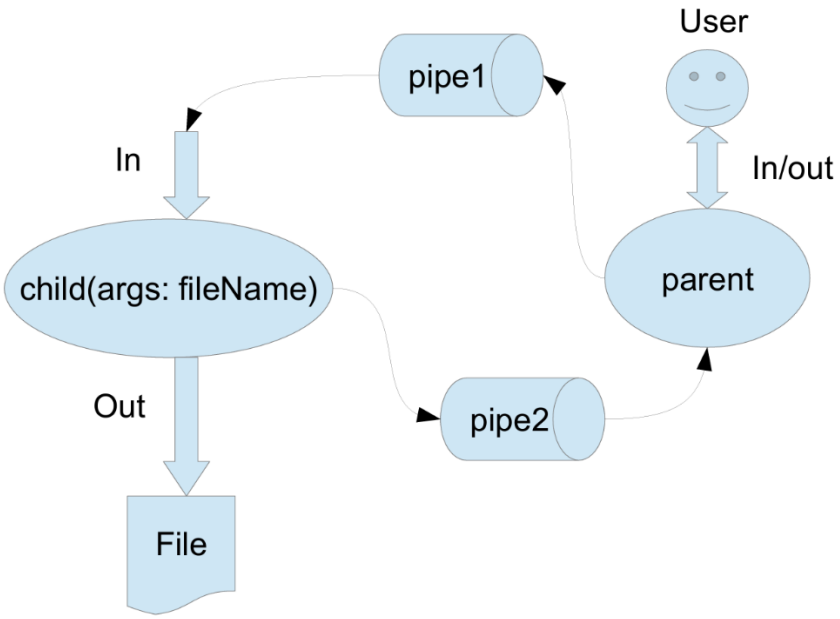
Оценка: _____

Дата: 10.10.26

Москва, 2026

Постановка задачи

Вариант 4:



Родительский процесс создает дочерний процесс. Первой строчкой пользователь в консоль родительского процесса пишет имя файла, которое будет передано при создании дочернего процесса. Родительский и дочерний процесс должны быть представлены разными программами. Родительский процесс передает команды пользователя через pipe1, который связан с стандартным входным потоком дочернего процесса. Дочерний процесс при необходимости передает данные в родительский процесс через pipe2. Результаты своей работы дочерний процесс пишет в созданный им файл. Допускается просто открыть файл и писать туда, не перенаправляя стандартный поток вывода.

Пользователь вводит команды вида: «число число число». Далее эти числа передаются от родительского процесса в дочерний. Дочерний процесс производит деление первого числа, на последующие, а результат выводит в файл. Если происходит деление на 0, то тогда дочерний и родительский процесс завершают свою работу. Проверка деления на 0 должна осуществляться на стороне дочернего процесса. Числа имеют тип float. Количество чисел может быть произвольным.

Общий метод и алгоритм решения

Использованные системные вызовы:

• pid_t fork(void); – создает новый (дочерний) процесс путем дублирования текущего процесса;

• int pipe(int pipefd[2]); – создает неименованный однонаправленный канал межпроцессного взаимодействия; возвращает дескриптор для чтения (pipefd[0]) и дескриптор для записи (pipefd[1]);

• int dup2(int oldfd, int newfd); – переназначает файловый дескриптор newfd на объект, на который указывает oldfd (используется для перенаправления стандартных потоков ввода/вывода);

• int execv(const char *pathname, char *const argv[]); – заменяет текущий образ памяти процесса новым исполняемым файлом с передачей аргументов командной строки;

• int open(const char *pathname, int flags, mode_t mode); – открывает или создает файл по указанному пути с заданными флагами доступа и правами;

• int close(int fd); – закрывает открытый файловый дескриптор;

• ssize_t read(int fd, void *buf, size_t count); – считывает до count байт данных из файлового дескриптора fd в буфер buf;

• ssize_t write(int fd, const void *buf, size_t count); – записывает до count байт данных из буфера buf в файловый дескриптор fd;

• pid_t waitpid(pid_t pid, int *status, int options); – приостанавливает выполнение родительского процесса до завершения дочернего процесса с идентификатором pid;

• void exit(int status); – немедленно завершает выполнение процесса с возвратом кода статуса ядру ОС.

Описание работы программы

Решение состоит из двух программ на языке Си: родительской (parent_program.c) и дочерней (child_program.c). **Родительская программа** считывает имя выходного файла из стандартного потока ввода (дескриптор 0) с помощью read(), сохраняя его в буфере filename (с удалением завершающего \n). Далее вызовами pipe() создаются два неименованных канала: pipe1 (для передачи данных потомку) и pipe2 (для приёма результатов от потомка). Вызовом fork() порождается дочерний процесс.

В ветке дочернего процесса (child == 0) с помощью dup2() стандартный поток ввода (дескриптор 0) перенаправляется на чтение из pipe1[0], а стандартный поток вывода (дескриптор 1) — на запись в pipe2[1]. Исходные дескрипторы каналов закрываются через close(), после чего вызывается execv("./child_program", args), который замещает образ процесса программой child_program и передаёт ей имя файла в качестве аргумента argv[1].

В ветке родительского процесса (default) закрываются неиспользуемые концы каналов pipe1[0] и pipe2[1]. Затем родитель в цикле считывает пользовательский ввод с клавиатуры через read() в буфер buffer. Если первый символ строки — \n (пустая строка), цикл прерывается. Иначе прочитанные байты пересылаются в канал pipe1[1] через write(), после чего родитель читает ответ из pipe2[0] и выводит его в стандартный поток вывода. Цикл также прерывается, если в ответе встречается подстрока "Division by zero\n" (проверка через strstr). По завершении цикла родитель закрывает pipe1[1] и pipe2[0] (отправляя EOF потомку) и ожидает завершения дочернего процесса через wait().

Дочерняя программа принимает имя файла через argv[1] и открывает (или создаёт) его системным вызовом open() с флагами O_WRONLY | O_CREAT | O_TRUNC | O_APPEND и правами доступа 0600. Затем в цикле читает данные из своего стандартного ввода (дескриптор 0, подключённый к pipe1) в буфер, завершая его нулевым символом. С помощью strtok() строка разбивается на токены по пробелам, табуляциям и переводам строки; каждый токен преобразуется в число типа float функцией strtof() и сохраняется в массив numbers (не более 256 элементов). Вычисляется результат последовательным делением: result = numbers[0], затем result /= numbers[i] для всех последующих чисел. Если очередной делитель равен нулю, в стандартный поток вывода (дескриптор 1, подключённый к pipe2) записывается сообщение "Division by zero\n", файл

закрывается, и процесс аварийно завершается через exit(1). Иначе результат форматируется в строку "Result = %f\n" через snprintf() и записывается как в выходной файл, так и в стандартный поток вывода. При ошибке записи в файл выводится сообщение об ошибке и процесс завершается. Когда родитель закрывает pipe1[1], чтение из канала возвращает 0, цикл завершается, файл закрывается через close(), и процесс успешно завершается.

Код программы

parent_program.c

```
#include <unistd.h>
#include <stdint.h>
#include <sys/wait.h>
#include <stdlib.h>
#include <string.h>

int main(int argc, char** argv) {

    char filename[1024];
    write(STDOUT_FILENO, "Enter filename: ", 16);
    ssize_t name = read(STDIN_FILENO, filename, sizeof(filename));
    if (name <= 0) {
        const char msg[] = "Error: failed to read filename\n";
        write(STDERR_FILENO, msg, sizeof(msg));
        exit(1);
    }
    if (filename[name - 1] == '\n') {
        filename[name - 1] = '\0';
        name--;
    }
    int pipe1[2];
    if (pipe(pipe1) == -1) {
        const char msg[] = "Error: failed to create pipe\n";
```

```
    write(STDERR_FILENO, msg, sizeof(msg));
    exit(1);
}

int pipe2[2];
if (pipe(pipe2) == -1) {
    const char msg[] = "Error: failed to create pipe\n";
    write(STDERR_FILENO, msg, sizeof(msg));
    exit(1);
}

const pid_t child = fork();

switch(child) {
case -1: {
                const char msg[] = "Error: failed to spawn new process\n";
                write(STDERR_FILENO, msg, sizeof(msg));
                exit(1);
        } break;

case 0: {
    close(pipe1[1]);
    close(pipe2[0]);

    dup2(pipe1[0], STDIN_FILENO);
    close(pipe1[0]);

    dup2(pipe2[1], STDOUT_FILENO);
    close(pipe2[1]);
```

```
    char *path = "./child_program";
    char *const args[] = {
        "./child_program",
        filename,
        NULL
    };

    int status = execv(path, args);
    if (status == -1) {
        const char msg[] = "Error: failed to exec\n";
        write(STDERR_FILENO, msg, sizeof(msg));
        exit(1);
    }
} break;

default: {
    close(pipe1[0]);
    close(pipe2[1]);

    char buffer[4096];
    ssize_t bytes;
    while (bytes = read(STDIN_FILENO, buffer, sizeof(buffer))) {
        if (bytes < 0) {
            const char msg[] = "Error: failed to read from stdin\n";
            write(STDERR_FILENO, msg, sizeof(msg));
            exit(1);
        } else if (buffer[0] == '\n') {
            break;
        }
```

```
            write(pipe1[1], buffer, bytes);

            bytes = read(pipe2[0], buffer, sizeof(buffer));
            if (bytes > 0) {
                write(STDOUT_FILENO, buffer, bytes);
                if (strstr(buffer, "Division by zero\n")) {
                    break;
                }
            }
        }

        close(pipe1[1]);
        close(pipe2[0]);

        wait(NULL);
    } break;
    }
    return 0;
}
```

child_program.c

```
#include <stdlib.h>
#include <unistd.h>
#include <fcntl.h>
#include <string.h>
#include <stdio.h>

int main(int argc, char **argv) {
    char buffer[4096];
    ssize_t bytes;
```

```
int file = open(argv[1], O_WRONLY | O_CREAT | O_TRUNC | O_APPEND, 0600);
if (file == -1) {
    const char msg[] = "Error: failed to open file\n";
    write(STDERR_FILENO, msg, sizeof(msg));
    exit(1);
}

while (bytes = read(STDIN_FILENO, buffer, sizeof(buffer) - 1)) {
    if (bytes < 0) {
                        const char msg[] = "Error: failed to read from stdin\n";
                        write(STDERR_FILENO, msg, sizeof(msg));
                        exit(1);
                }

    buffer[bytes] = '\0';

    float numbers[256];
    int count = 0;
    char *token = strtok(buffer, " \t\n");
    while (token != NULL && count < 256) {
        numbers[count++] = strtof(token, NULL);
        token = strtok(NULL, " \t\n");
    }

    float result = numbers[0];
    for (int i = 1; i < count; ++i) {
        if (numbers[i] == 0) {
            const char msg[] = "Division by zero\n";
            write(STDOUT_FILENO, msg, sizeof(msg));
```

```
            close(file);
            exit(1);
        }
        result /= numbers[i];
    }

    char res[64];
    int result_string = snprintf(res, sizeof(res), "Result = %f\n", result);
    int written = write(file, res, result_string);
    write(STDOUT_FILENO, res, result_string);

    if (written != result_string) {
        const char msg[] = "Error: failed to write to file\n";
        write(STDERR_FILENO, msg, sizeof(msg));
        close(file);
        exit(1);
    }
  }
  close(file);
  return 0;
}
```

Протокол работы программы

Тестирование:

```
awareness@mine:~/process_lab$ cc -o child_program child_program.c
awareness@mine:~/process_lab$ cc -o parent_program parent_program.c
awareness@mine:~/process_lab$ ./parent_program
Enter filename: test.txt
276 -183
Result = -1.508197
8238838 387272.3232
```

Result = 21.274017

28383 28382 88

Result = 0.011364

99 0 33

Division by zero

awareness@mine:~/process_lab$ cat test.txt

Result = -1.508197

Result = 21.274017

Result = 0.011364

awareness@mine:~/process_lab$ ./parent_program

Enter filename: text

77 11 -7

Result = -1.000000

374.232 3.5

Result = 106.923424

0 0

Division by zero

awareness@mine:~/process_lab$

Вывод

В ходе выполнения лабораторной работы были получены практические навыки управления процессами в ОС Linux и организации двунаправленного межпроцессного взаимодействия с помощью неименованных каналов (pipe). Были изучены и применены на практике системные вызовы стандарта POSIX: fork, execv, wait, pipe, dup2, open, close, read и write. Основная трудность при реализации заключалась в отказе от стандартной библиотеки <stdio.h> и потокового ввода-вывода: потребовалось самостоятельно реализовать чтение данных из файлового дескриптора порциями фиксированного размера, разбор строки на числа с помощью strtok и strtof, а также форматирование результата в строку через snprintf. Таким образом, работа позволила на практике закрепить понимание создания процессов, замены образа процесса, перенаправления стандартных потоков и передачи данных между процессами через каналы.